# OS Assignment

Name: Annanya Sharma

SRN: PES1UG24CS069

## PHASE 1:

1A)

```
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
```

1B)

```
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$
```

## Phase 2:

2A)

```
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B)

```
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ find .pes/objects -type f
.pes/objects/f8/b93f79591de4e825e2072f77789783d428dceb30c5cc2a8e7484465a0ceab9
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ xxd .pes/objects/f8/b93f79591de4e825e2072f77789783d428dceb30c5cc2a8e7484465a0ceab9 | head -20
00000000: 7472 6565 2034 3800 3130 3036 3434 2066  tree 48.100644 f
00000010: 696c 652e 7478 7400 aaaa aaaa aaaa aaaa  ile.txt.........
00000020: aaaa aaaa aaaa aaaa aaaa aaaa aaaa aaaa  ................
00000030: aaaa aaaa aaaa aaaa                      ........
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$
```

## Phase 3:

3A)

```
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ echo "hello" > file1.txt
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ echo "world" > file2.txt
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ ./pes add file1.txt file2.txt
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.c
  untracked:  Makefile
  untracked:  tree.c
  untracked:  index.h
  untracked:  pes.c
  untracked:  tree.h
  untracked:  index.c
  untracked:  test_objects.c
  untracked:  object.c
  untracked:  .gitignore
  untracked:  README.md
  untracked:  pes.h
  untracked:  test_tree.c
  untracked:  commit.h
  untracked:  test_sequence.sh
```

3B)

```
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776590502 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776590506 6 file2.txt
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$
```

# Phase 4:

4A)

```
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ ./pes log
commit 3c6f620d3096539c879b26603d7284569720bb5515c384c0063ed1cd8ccfef8c
Author: Annanya Sharma <PES1UG24CS069>
Date:   1776604407

    Add farewells:insta !

commit ae1da269cd9a305164a01358051e983feb974c55b65566c0bdbedf8d1a9af025
Author: Annanya Sharma <PES1UG24CS069>
Date:   1776604388

    Add world

commit 26a536e2b0418948b33f5063c5843146a16797d604e15da29eee41749c034b6e
Author: Annanya Sharma <PES1UG24CS069>
Date:   1776604370

    Initial committial commitES1A
```

4B)

```
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/26/a536e2b0418948b33f5063c5843146a16797d604e15da29eee41749c034b6e
.pes/objects/3c/6f620d3096539c879b26603d7284569720bb5515c384c0063ed1cd8ccfef8c
.pes/objects/50/da219c4c66a82940f7a8555908504a9bb1ede3773efe8669ad72f1d54966b3
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7c/83d6df2d741034233b181c4bfa7a6e64a49ec875cc62a850886d11af72429c
.pes/objects/ae/1da269cd9a305164a01358051e983feb974c55b65566c0bdbedf8d1a9af025
.pes/objects/c1/5daab6ccf47288c923fa5384e07d0e7c63a4f443a1a77c77d51e67b257e822
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

4C)

```
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ cat .pes/refs/heads/main
3c6f620d3096539c879b26603d7284569720bb5515c384c0063ed1cd8ccfef8c
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$
```

FULL INTEGRATION TEST:

```
vboxuser@unit-4: ~/PES1UG24CS069-pes-vcs
Author: Annanya Sharma <PES1UG24CS069>
Date:   1776606212

    Initial committial commitES1A

vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 174209d3f133... Initial commit

Log after first commit:
commit 174209d3f133b55d52047b1cb8a839e7eabc7ad00ea8003582e1fdf6a37c6830
Author: Annanya Sharma <PES1UG24CS069>
Date:   1776606271

    Initial commit


--- Second Commit ---
Committed: 60c151ac70c9... Update file.txt

--- Third Commit ---
Committed: 60458ed3123c... Add farewell

--- Full History ---
commit 60458ed3123ca0dec6ca556cf4dab6dda7b5bdc6ff1e5de4e1467bb013cd1829
```

```
vboxuser@unit-4: ~/PES1UG24CS069-pes-vcs
--- Second Commit ---
Committed: 60c151ac70c9... Update file.txt

--- Third Commit ---
Committed: 60458ed3123c... Add farewell

--- Full History ---
commit 60458ed3123ca0dec6ca556cf4dab6dda7b5bdc6ff1e5de4e1467bb013cd1829
Author: Annanya Sharma <PES1UG24CS069>
Date:   1776606271

    Add farewells:insta !

commit 60c151ac70c9838e4a34b15b3db1207a39854fd5c0b65faec3e92d1c23fad633
Author: Annanya Sharma <PES1UG24CS069>
Date:   1776606271

    Update file.txtnsta !

commit 174209d3f133b55d52047b1cb8a839e7eabc7ad00ea8003582e1fdf6a37c6830
Author: Annanya Sharma <PES1UG24CS069>
Date:   1776606271

    Initial committial commitES1A


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
60458ed3123ca0dec6ca556cf4dab6dda7b5bdc6ff1e5de4e1467bb013cd1829

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/17/4209d3f133b55d52047b1cb8a839e7eabc7ad00ea8003582e1fdf6a37c6830
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/60/458ed3123ca0dec6ca556cf4dab6dda7b5bdc6ff1e5de4e1467bb013cd1829
.pes/objects/60/c151ac70c9838e4a34b15b3db1207a39854fd5c0b65faec3e92d1c23fad633
.pes/objects/84/c5fed72c2dd9a3130e2e9b98033827f62793fb80d0295ccedf5fc7972d96eb
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/de/34fb483a5c16ce29ec3195dd505dfda7e35cc5fe16a99da4b2455b332a93cf
.pes/objects/ec/0af37b159ae92a5850ceaa6e497860729d6158aab2c6833b6068a1e3c5e8ef

=== All integration tests completed ===
vboxuser@unit-4:~/PES1UG24CS069-pes-vcs$
```

# PHASE 5:

### Q5.1 — Implementing `pes checkout <branch>`

A branch in this system is stored as a file inside `.pes/refs/heads/` containing a commit hash, and HEAD points to one such reference. To implement `pes checkout <branch>`, the following steps are required:

- Update HEAD: Modify `.pes/HEAD` to point to `ref: refs/heads/<branch>`.
- Read target commit: Open `.pes/refs/heads/<branch>` and obtain the commit hash.
- Load snapshot: Read the commit object and extract its tree (project snapshot).
- Update working directory:
    - Create or overwrite files to match the tree contents.
    - Delete files that are not present in the target tree.
- Update index: Synchronize the index with the new tree so staging reflects the checked-out state.

This operation is complex because it must:

- Safely replace the working directory contents,
- Handle file additions, deletions, and overwrites,
- Prevent accidental loss of uncommitted changes.

### Q5.2 — Detecting a dirty working directory

A "dirty" working directory means files have been modified but not committed. This must be detected before checkout to avoid data loss.

Detection can be done using the index and object store:

- Compare working directory vs index:
    - For each tracked file in the index:

        - Check file metadata (size, modification time), or recompute hash.
        - If it differs then file is modified (dirty).
- Compare current state vs target branch:
    - Load the target branch's tree from the object store.
    - Check if the same file differs between:
        - current index
        - target tree
- Conflict condition:
    - If a file is:
        - modified in working directory
        - different in target branch then checkout must refuse

This ensures no uncommitted changes are overwritten.

**Q5.3 — Detached HEAD and commits**

A detached HEAD occurs when `.pes/HEAD` directly contains a commit hash instead of a branch reference.

- What happens during commits:
    - New commits are created normally.
    - Each commit points to its parent.
    - However, no branch reference is updated.
- Result:
    - These commits form a chain but are not attached to any branch
    - They become orphaned when switching away.
- Recovery:
    - Commits still exist in the object store.
    - User can recover by:
        - Creating a new branch manually pointing to the commit hash.

# Phase 6:

### Q6.1 — Garbage collection of unreachable objects

Over time, objects (commits, trees, blobs) may become unreachable if no branch references them.

Algorithm:

1. Collect root references:
    a. All files in .pes/refs/heads/
    b. Current HEAD
2. Traverse reachable objects:
    a. Start from each commit:
        i. Mark commit as reachable
        ii. Follow parent pointer
        iii. Visit associated tree
    b. From trees:
        i. Recursively visit subtrees and blobs
3. Track visited objects:
    a. Use a hash set for efficiency
    b. Prevent revisiting same objects
4. Delete unreachable objects:
    a. Scan .pes/objects
    b. Remove objects not in reachable set

Complexity:

- With 100,000 commits and 50 branches:
    - Due to shared history, traversal ≈ 100,000 commits, not 5 million
    - Trees and blobs increase count, but sharing keeps it manageable

**Q6.2 — Dangers of concurrent garbage collection**

Running garbage collection during a commit can cause race conditions.

Problem scenario:

- Commit process:
  - Writes blob/tree objects
  - (Not yet referenced by any commit)
- At the same time, GC runs:
  - Sees these objects as unreachable
  - Deletes them
- Commit continues:
  - Tries to reference deleted objects → corruption or failure

How real systems avoid this:

- Atomic writes:
  - Objects written completely before becoming visible
- Temporary files + rename:
  - Ensures partially written objects are not exposed
- Locking mechanisms:
  - Prevent GC from running during critical operations
- Grace periods / delayed deletion:
  - Recently created objects are not immediately removed